KARNAVATI UNIVERSITY
UNIVERSITY
NAAC GRADE A+

KARNAVATI UNIVERSITY
UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

"Autonomous Deep Research AI Agent"

Submitted as a part of the course

AGENTIC AI

Submitted By

Group Name: Prompt Pirates

Student Name	Enrollment No.
Utpal Sakhiya	202307020101
Vyom Patel	202307020084
Harshil Dhamat	202307020033
Poojan Shihora	202307020111

Program: B.Tech CSE-AIML
Semester: VI
Section: _____

Under the Guidance of

Prof. Vaishnavi Chauhan
Assistant Professor
Department of Computer Science & Information Technology

Karnavati University, Gandhinagar

Academic Year: 2026–27

CERTIFICATE

This is to certify that the mini project entitled **"Autonomous Deep Research AI Agent"** has been successfully completed by the following students of B.Tech CSE-AIML Semester VI, Karnavati University, as a part of the course **Agentic AI** during the academic year 2026–27.

Sr. No.	Student Name	Enrollment No.
1	Utpal Sakhiya	202307020101
2	Vyom Patel	202307020084
3	Harshil Dhamat	202307020033
4	Poojan Shihora	202307020111

The project has been carried out under the guidance and supervision of the faculty member. The work presented in this report is the original work carried out by the students as part of their mini project.

Faculty Guide
Name: Prof. Vaishnavi Chauhan
Signature: _____
Date: _____

DECLARATION

We hereby declare that the mini project entitled **"Autonomous Deep Research AI Agent"** submitted as part of the course **Agentic AI** is our original work carried out under the guidance of the faculty member.

We further declare that this work has not been submitted previously to any other institution or university for the award of any degree, diploma, or academic certification.

All research papers, datasets, frameworks, APIs, libraries, websites, and other resources used during the development of the project have been appropriately acknowledged and referenced.

Sr. No.	Student Name	Enrollment No.
1	Utpal Sakhiya	202307020101
2	Vyom Patel	202307020084
3	Harshil Dhamat	202307020033
4	Poojan Shihora	202307020111

Date: _____
Place: Gandhinagar

ACKNOWLEDGEMENT

We, the members of team Prompt Pirates, express our sincere gratitude to our faculty guide, Prof. Vaishnavi Chauhan, Assistant Professor, Department of Computer Science & Information Technology, for continuous guidance, constructive feedback and encouragement throughout the development of this mini project. The guidance on agent design, evaluation and honest reporting of results shaped both the system and this report.

We thank the Department of Computer Science & Information Technology and the Unitedworld Institute of Technology, Karnavati University, Gandhinagar, for introducing the Agentic AI course and for providing the academic environment, laboratory infrastructure and internet access needed to build and test the project.

We also acknowledge the open-source communities behind LangGraph, LangChain, Pydantic, trafilatura, Starlette, React and pytest, and the providers of the Google Gemini API and the Tavily Search API, whose free tiers made live experimentation possible. Finally, each member of the team contributed to the design, implementation, testing and documentation of the system, and we thank one another for the collaboration.

ABSTRACT

Answering a complex research question normally requires a person to break it into searches, read many web pages, write a summary with citations and then check what is still missing. A single prompt to a large language model (LLM) cannot do this reliably: it answers from its training data, cannot fetch new evidence and does not review its own answer. This project presents the **Autonomous Deep Research AI Agent**, a single-agent, graph-based system that turns one research question into a cited Markdown report.

The agent is implemented as a LangGraph state machine with five role-specialised nodes. A planner decomposes the question into 3-6 sub-queries; a searcher calls the Tavily Search API and scrapes the top pages; a synthesizer writes a draft that may cite only the retrieved, numbered sources; a reflector scores the draft from 1 to 10 and proposes follow-up searches; and a router loops back to the searcher until the draft scores 8 or more, three review rounds are reached, or a token budget is spent. A finalizer removes any citation that does not match a retrieved source and saves the report. Google Gemini (`gemini-3.5-flash-lite`) is the LLM, and Pydantic structured output drives every control-flow decision. The system offers a React web interface with live progress, a command-line interface and a Streamlit interface.

All 104 automated tests passed (60 pytest, 44 Vitest). In 3 live runs, 3 completed; the average run took 206 s and 25,394 tokens, 63% of pages were scraped successfully, and every citation in the final reports pointed to a retrieved source. The reflection loop triggered a second search round in 1 of 3 runs, raising the self-assessed score from 6/10 to 9/10. The project shows that a small, explicit plan-act-reflect loop with grounded citations is a practical way to automate first-draft web research.

TABLE OF CONTENTS

CERTIFICATE 2

DECLARATION 3

ACKNOWLEDGEMENT 4

ABSTRACT 5

LIST OF FIGURES 9

LIST OF TABLES 10

CHAPTER 1 – INTRODUCTION 11

1.1 Introduction 11

1.2 Background 12

1.3 Problem Statement 12

1.4 Motivation 12

1.5 Objectives 13

1.6 Scope of the Project 13

CHAPTER 2 – LITERATURE REVIEW 15

2.1 Existing Systems and Research 15

2.2 Comparative Analysis 15

2.3 Research Gap 16

CHAPTER 3 – PROPOSED SYSTEM 17

3.1 Proposed Solution 17

3.2 System Requirements 18

3.2.1 Hardware Requirements 18

3.2.2 Software Requirements 18

3.3 Functional Requirements 18

3.4 Non-Functional Requirements 19

CHAPTER 4 – AGENT ARCHITECTURE 20

4.1 Overall System Architecture 20

4.2 Architecture Components 20

4.3 Agent Roles and Responsibilities 21
Component Responsibilities 21
Role-Specialised Nodes 21
4.4 Agent Workflow 22
CHAPTER 5 – AGENTIC AI CONCEPTS USED 24
5.1 ReAct (Reason → Act → Observe) 24
5.2 Planning 24
5.3 Tool Calling 25
5.4 Memory 26
5.5 Retrieval-Augmented Generation 26
5.6 Multi-Agent System 27
5.7 Human-in-the-Loop 27
CHAPTER 6 – METHODOLOGY 28
6.1 Input Processing 28
6.2 Task Understanding 28
6.3 Task Planning 28
6.4 Reasoning and Decision Making 28
6.5 Tool Selection 29
6.6 Task Execution 29
6.7 Observation and Feedback 29
6.8 Final Response Generation 29
CHAPTER 7 – IMPLEMENTATION 30
7.1 Project Structure 30
7.2 Agent Implementation 30
7.3 Tool Implementation 31
7.4 Memory Implementation 32
7.5 RAG Implementation 33
7.6 User Interface 34
CHAPTER 8 – RESULTS AND OUTPUT 38

8.1 Test Cases 38
8.2 Sample Outputs 39
Test Case 1: Web UI run (technical question) 39
Test Case 2: CLI run (EV battery comparison) 41
Test Case 3: CLI run (nuclear power and net zero) 41
8.3 Agent Execution Trace 42
CHAPTER 9 – PERFORMANCE EVALUATION 44
CHAPTER 10 – CHALLENGES AND LIMITATIONS 45
CHAPTER 11 – FUTURE SCOPE 46
CHAPTER 12 – CONCLUSION 47
REFERENCES 48
Research Papers 48
Books 48
Websites, Documentation, APIs and Libraries 48
APPENDIX 50
Appendix A – Source Code 50
Appendix B – Additional Screenshots 50
Appendix C – Sample Prompts 51
Appendix D – Test Data 51

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Overall System Architecture	20
Figure 2	Agent Workflow	23
Figure 3	Home page of the web interface	34
Figure 4	Input interface with a research question entered	35
Figure 5	Agent execution: run started, planner decomposing the question	35
Figure 6	Tool execution: live sources retrieved and scraped by the searcher	36
Figure 7	Reflection loop: draft scored 6/10, agent searching again	36
Figure 8	Final output: finished report with score, iterations and sources	37
Figure 9	Automated test results (pytest and Vitest)	39
Figure 10	Sample output: report body with inline citations	40
Figure 11	Sample output: first draft compared with the revised report	40
Figure 12	CLI run: EV battery comparison	41
Figure 13	CLI run: nuclear power and net zero	42
Figure 14	Report library with the three live-run reports and the sources panel	50

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Literature survey of related systems and research	15
Table 2	Comparison of the existing approach and the proposed system	15
Table 3	Hardware requirements	18
Table 4	Software requirements and technologies used	18
Table 5	Functional requirements	19
Table 6	Architecture components	21
Table 7	Component responsibilities	21
Table 8	Role-specialised nodes of the agent graph	21
Table 9	Agentic AI concepts and their implementation status	24
Table 10	Tools used by the agent	25
Table 11	Test cases	38
Table 12	Results of the live research runs	39
Table 13	Performance evaluation	44
Table 14	Sample prompts used in the system	51
Table 15	Test data: research questions	51

CHAPTER 1 – INTRODUCTION

1.1 Introduction

Artificial Intelligence (AI) is the field of building computer systems that perform tasks which normally need human intelligence, such as perception, reasoning, learning and decision making. Russell and Norvig describe AI as the study of agents that receive percepts from an environment and perform actions [8].

Generative AI is the branch of AI that produces new content (text, images, code) by learning patterns from large datasets. Large language models (LLMs) such as Google Gemini are the main example: given a prompt, they generate fluent text, but on their own they answer only from what they learned during training.

Agentic AI places an LLM inside a control loop so that it can pursue a goal over several steps: it decides what to do next, calls tools, observes the results and adjusts. The difference between a traditional chatbot and an AI agent is summarised below, with how each property appears in this project:

- **Autonomy:** a chatbot replies once per message; our agent runs the whole research process (plan, search, write, review, revise) from a single question without further input.
- **Planning:** the planner node decomposes the question into 3-6 focused web search queries before any searching starts.
- **Reasoning:** the reflector node critiques each draft against the plan, lists concrete gaps and decides which follow-up searches would close them.
- **Tool usage:** the agent uses a web search API (Tavily) and a web page scraper to gather evidence that the LLM does not have.
- **Memory:** a shared state object keeps the plan, every source, every draft and every critique for the duration of a run (short-term working memory).
- **Goal-oriented execution:** the run continues until a measurable goal is met (a review score of at least 8/10) or a safety limit is reached.

This project therefore applies agentic AI to a concrete problem: producing a well-cited research report from live web sources.

1.2 Background

Desk research is a routine task for students, analysts and journalists. The conventional process is manual: type several searches into a search engine, open and read many pages, copy relevant facts, write a summary and add references. Search engines return ranked links, not answers, so the synthesis and the checking of coverage remain with the person.

Chat assistants built on LLMs produce fluent answers instantly, but a plain LLM call has three weaknesses for research: its knowledge stops at its training cut-off, it can invent facts and references (hallucination), and it never checks whether its own answer covered the whole question. Retrieval-augmented generation [2] reduces the first two problems by giving the model retrieved documents, and self-feedback methods such as Reflexion [4] and Self-Refine [5] address the third by letting the model critique and improve its own output.

Agentic AI combines these ideas. An agent can plan its searches, use tools to retrieve fresh evidence, write with citations restricted to that evidence, and then evaluate the result and search again for what is missing. This project implements exactly that loop with explicit, testable control flow.

1.3 Problem Statement

The objective of this project is to develop an Agentic AI-based system that takes a complex research question, autonomously plans and performs web searches, reads the retrieved pages and writes a Markdown report in which every claim is cited to a page that was actually fetched. Existing systems either return unsynthesised links (search engines) or produce uncited answers from static training data in a single pass (LLM chatbots). Therefore, there is a need for a system that can plan, retrieve, write, critically review its own draft and search again until the report is complete, while keeping its citations grounded and its cost bounded.

1.4 Motivation

- **Why this problem:** research questions such as technology comparisons or market overviews need many sources, and collecting them by hand takes hours.

- **Why automation:** the steps (search, read, summarise, check coverage) are repetitive and well defined, which makes them suitable for an automated pipeline.
- **Why Agentic AI:** the task needs decisions that depend on intermediate results (which gaps remain, what to search next), which a fixed script or a single LLM call cannot make.
- **Who benefits:** students preparing assignments, analysts writing briefings and anyone who needs a cited first draft to verify and extend.
- **Limitations that motivated us:** uncited or fabricated references in chatbot answers, and the lack of any self-check on coverage in one-shot generation.

1.5 Objectives

1. To design an Agentic AI system that converts a single research question into a cited Markdown research report.
2. To implement autonomous task planning (question decomposition into 3-6 sub-queries) and execution with LangGraph.
3. To integrate a web search API (Tavily) and a robust web scraper as tools, with retries and graceful failure handling.
4. To implement a reflection loop in which the agent scores its own draft and performs follow-up searches until a quality threshold (8/10) or a limit (3 rounds, 400,000 tokens) is reached.
5. To ground every citation in retrieved sources and remove any citation that does not match one.
6. To evaluate the system with automated test cases and live runs, and to analyse its effectiveness and limitations using measured values only.

1.6 Scope of the Project

- **Target users:** students, researchers and analysts who need a first, cited draft on a topic.
- **Supported tasks:** open-ended research questions in English, such as comparisons, explanations of how something works, recent developments and market overviews.

- **Functional scope:** planning, web search, page scraping, cited report writing, self-review with follow-up searches, citation validation, saving reports, and browsing past reports in a library.
- **Technologies covered:** Python, LangGraph, Google Gemini, Tavily, httpx, trafilatura, Pydantic, Starlette, React/TypeScript, Streamlit, pytest and Vitest.
- **Expected usage:** run locally through the web UI (`python -m app.web`), the CLI (`python -m app.cli`) or Streamlit, one research job at a time.
- **Current limitations:** no long-term memory or vector database, no human approval step, paywalled or JavaScript-only pages are not scraped, the quality score is the model's own assessment, and free-tier API quotas limit throughput.

CHAPTER 2 – LITERATURE REVIEW

2.1 Existing Systems and Research

The project builds on research into retrieval-augmented generation, tool-using agents and self-feedback. Five representative works are compared in Table 1.

Table 1: Literature survey of related systems and research

Sr. No.	Paper/System	Year	Approach	Key Finding	Limitation
1	WebGPT (Nakano et al.) [1]	2021	GPT-3 fine-tuned to search and browse a text web browser and answer with collected references	Answers backed by collected references were preferred by human raters over the baseline model	Needs fine-tuning with human feedback; single pass with no explicit coverage check
2	RAG (Lewis et al.) [2]	2020	Dense retriever over Wikipedia combined with a seq2seq generator	Retrieval improves knowledge-intensive QA and makes outputs more specific and factual	Static index; one retrieval pass; no self-review of the answer
3	ReAct (Yao et al.) [3]	2023	LLM interleaves reasoning traces with actions such as Wikipedia API calls	Outperforms reasoning-only and acting-only prompting on QA and decision-making tasks	Open-ended prompt loop; stopping and cost are hard to control
4	Reflexion (Shinn et al.) [4]; Self-Refine (Madaan et al.) [5]	2023	The model critiques its own output in natural language and retries using the feedback	Self-feedback improves results without updating model weights	Relies on the quality of the self-evaluation; more LLM calls
5	STORM (Shao et al.) [6]	2024	Researches a topic by asking questions from several perspectives, then writes a cited, Wikipedia-like article	Articles judged better organised and broader in coverage than baselines	Long multi-step pipeline; inherits bias of retrieved sources

2.2 Comparative Analysis

Table 2 compares a conventional single-prompt LLM assistant (combined with manual web search) with the proposed agent. Only features that are relevant to this project are listed.

Table 2: Comparison of the existing approach and the proposed system

Feature	Existing System (single LLM prompt + manual search)	Proposed System
Autonomous Execution	No; the user drives every step	Yes; one question runs the whole plan-search-write-review loop
Task Planning	None; one prompt, one answer	Planner decomposes the question into 3-6 sub-queries
Reasoning	Implicit, single pass	Reflector critiques coverage, support, accuracy and depth; router decides next step
Tool Usage	None (answers from training data)	Tavily web search and a robots.txt-aware scraper

Feature	Existing System (single LLM prompt + manual search)	Proposed System
Memory	Chat history only	Shared run state with accumulating sources, drafts and critiques
RAG	No	Web RAG: only retrieved, numbered sources may be cited
Citations	Often missing or fabricated	Sources list generated in code; invalid [n] removed
Multi-Agent Support	No	No (single agent with role-specialised nodes)
Human Intervention	Needed at every step	Only to ask the question and optionally stop the run

2.3 Research Gap

The surveyed work shows that retrieval improves factuality [2], that tool use interleaved with reasoning helps agents act [3], [6], and that self-feedback improves outputs [4], [5]. However, the following gaps remain for an everyday research assistant:

- Many agent loops are open-ended prompts, so stopping, cost and failure behaviour are hard to predict or test.
- Generated reports may cite URLs that were never retrieved, because the model writes the reference list itself.
- One-pass RAG does not check coverage: if the first retrieval misses an aspect of the question, the answer silently omits it.

This project addresses these gaps with an explicit LangGraph state machine whose decisions are made by validated structured output and tested by unit tests, a reference list generated in code from the retrieved sources, a citation validator, and a reflection loop that turns detected gaps into follow-up searches with hard limits on rounds and tokens.

CHAPTER 3 – PROPOSED SYSTEM

3.1 Proposed Solution

The proposed solution is a single autonomous research agent built as a LangGraph StateGraph. Its components are described below; Chapter 4 shows the architecture.

- **User interaction:** the user types a question (1-500 characters) in the React web UI, the CLI or the Streamlit app, then watches progress live and reads the final report.
- **Agent:** a compiled LangGraph graph with five nodes (planner, searcher, synthesizer, reflector, finalizer) and one conditional router. Each node is a pure function that reads the shared state and returns a partial update.
- **LLM:** Google Gemini (`gemini-3.5-flash-lite`) accessed through langchain-google-genai, with a client-side rate limiter (4 requests/minute), a daily request cap and automatic waiting on HTTP 429 quota errors.
- **Planning:** the planner converts the question into 3-6 non-overlapping search queries, validated by a Pydantic schema.
- **Reasoning:** the reflector reviews each draft for coverage, citation support, accuracy and depth, returns a score, a list of gaps and up to three follow-up queries.
- **Tools:** Tavily web search and a web page scraper (httpx, trafilatura, BeautifulSoup) that respects robots.txt.
- **Memory:** short-term working memory only: the ResearchState object holds the plan, sources, drafts and critiques during a run.
- **Knowledge base:** none is pre-built; the live web is the knowledge source for every run.
- **Retrieval:** search results are scraped, cleaned and truncated to 6,000 characters per page, then given to the LLM as numbered sources.
- **Execution:** LangGraph executes the nodes in order and follows the router's decision to loop or finish; the web server runs the graph in a worker thread and streams events.
- **Final response:** a Markdown report with an executive summary, sections, [n] citations and a Sources list generated from the retrieved URLs, saved to reports/.

3.2 System Requirements

3.2.1 Hardware Requirements

Table 3 lists the hardware needed. Because the LLM runs in the cloud, no GPU is required. The values in brackets describe the machine used for testing.

Table 3: Hardware requirements

Component	Requirement
Processor	Any 64-bit dual-core CPU (tested: 12th Gen Intel Core i5-1240P)
RAM	4 GB minimum, 8 GB recommended (tested: 8 GB)
Storage	About 1 GB free: Python environment about 0.6 GB, Node.js packages about 0.25 GB (only needed to rebuild the UI), reports a few KB each
GPU	Not required; LLM inference runs on Google's servers through the Gemini API
Internet Connection	Required for the Gemini API, the Tavily API and scraping web pages

3.2.2 Software Requirements

Table 4 lists the software actually used, with the versions pinned in requirements.txt and package.json.

Table 4: Software requirements and technologies used

Software/Technology	Version/Specification
Operating System	Windows 11 (tested); any OS with Python 3.11+
Programming Language	Python 3.11+ (tested 3.13.14); TypeScript 5.6 for the web UI
IDE/Environment	Visual Studio Code 1.143, Python venv, Node.js 24 (UI build only)
Agent Framework	LangGraph 1.2.14 with langchain-core 1.6.9
LLM	Google Gemini gemini-3.5-flash-lite via langchain-google-genai 3.2.0
Vector Database	Not used
Embedding Model	Not used
Frontend	React 18.3.1, Vite 5.4, Tailwind CSS 3.4, shadcn/ui (Radix), Zustand, TanStack Query; Streamlit 1.65 and a Rich 15 CLI
Backend / Server	Starlette 1.7.0 on Uvicorn 0.54.0; Pydantic 2.14.0 for validation
APIs/Tools	Tavily Search API (tavily-python 0.8.5); httpx 0.28.1; trafilatura 2.3.1; BeautifulSoup 4.15.0
Testing	pytest 9.1.1; Vitest 2.1.9 with Testing Library

3.3 Functional Requirements

The functional requirements and the module that implements each are listed in Table 5.

Table 5: Functional requirements

ID	Requirement	Implemented in
FR1	Accept a research question of 1-500 characters (web, CLI, Streamlit)	app/web.py, app/cli.py
FR2	Generate a plan of 3-6 unique search queries	nodes/planner.py, state.Plan
FR3	Search the web and scrape the top new pages for each query	nodes/searcher.py, tools/
FR4	Write a Markdown report citing only retrieved, numbered sources	nodes/synthesizer.py
FR5	Review the draft, score it and propose follow-up queries	nodes/reflector.py
FR6	Loop until score ≥ 8, 3 rounds, 400,000 tokens or no follow-ups	graph.route_after_reflection
FR7	Remove invalid citations and save the report (and first draft)	nodes/finalizer.py
FR8	Stream live progress and allow the user to stop a run	app/web.py, app/frontend
FR9	List, open, compare and download saved reports	app/web.py, app/frontend
FR10	Handle failed searches, scrapes, invalid JSON and quota errors without crashing	searcher.py, scrape.py, llm.py

3.4 Non-Functional Requirements

- **Accuracy:** every claim must carry a citation to a retrieved source; the Sources list is generated in code, and invalid citation numbers are removed by the finalizer. The synthesizer must write "insufficient evidence" instead of guessing.
- **Reliability:** a failed search is logged and skipped, a failed scrape returns None and falls back to the search snippet, structured output is retried once, and LangGraph's recursion limit (25) backs up the loop limits.
- **Performance:** pages are scraped in parallel (6 workers) with a 10 s timeout; a run is bounded by 3 review rounds and 400,000 tokens.
- **Scalability:** the web server deliberately runs one job at a time to stay within the free-tier LLM quota; the graph itself is stateless between runs.
- **Security:** API keys are read only from the git-ignored .env file; report names are validated with a strict pattern so paths such as ../../.env are rejected; Markdown is sanitised in the UI (rehype-sanitize).
- **Usability:** a live pipeline view, sources panel, keyboard shortcuts, a report reader with table of contents, citation previews, Markdown download and draft comparison.
- **Maintainability:** one node per file (each under 150 lines), all prompts in one module, type hints and docstrings throughout, and 104 automated tests.

CHAPTER 4 – AGENT ARCHITECTURE

4.1 Overall System Architecture

Figure 1 shows the actual architecture in four layers: the user interfaces, the Starlette web server, the LangGraph agent core with its shared state and LLM helpers, and the external services (Gemini, Tavily, public web pages) plus local report storage. The agent loop corresponds to: User → Input validation → Planner (planning) → Searcher (tool use) → Synthesizer → Reflector (reasoning / observation) → Router (decision) → either another search round or the Finalizer → Final report.

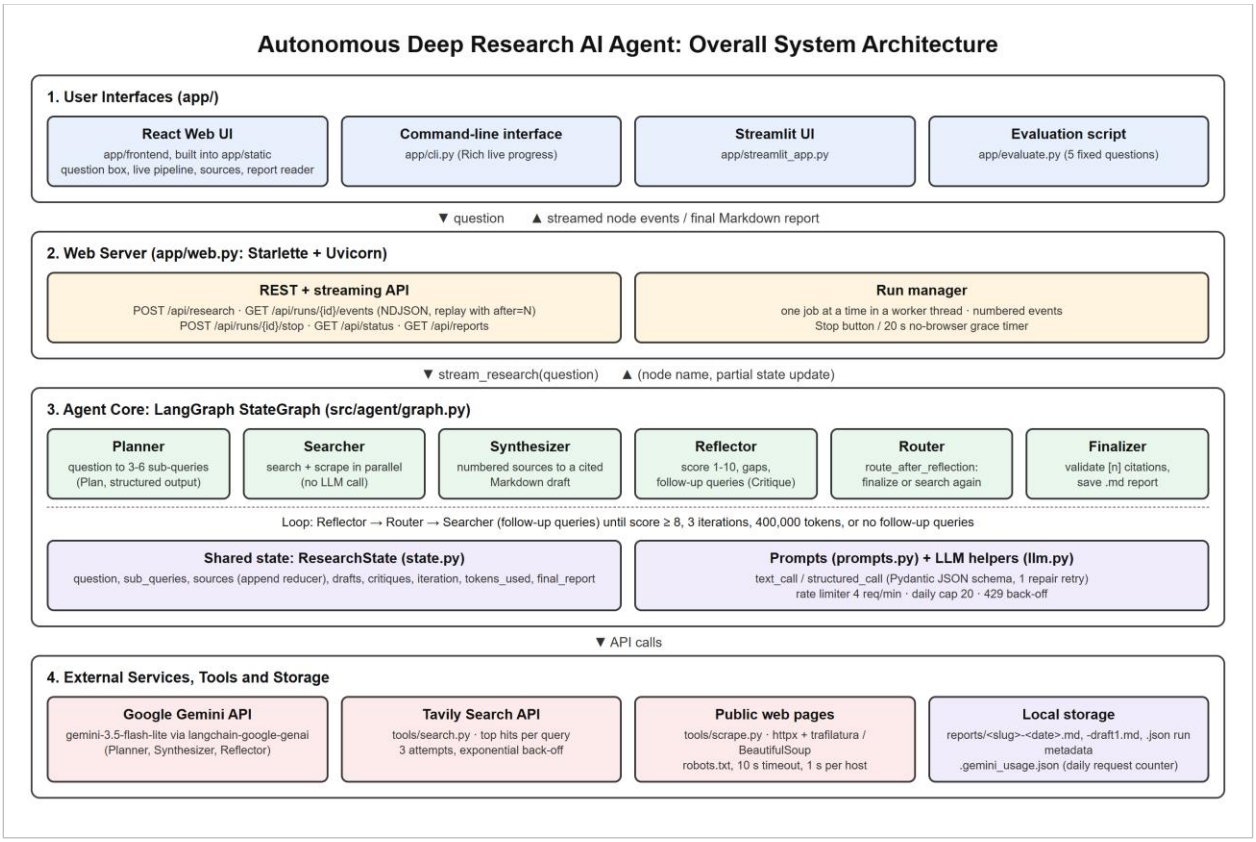


Figure 1: Overall System Architecture

4.2 Architecture Components

Each major component is explained in Table 6.

Table 6: Architecture components

Component	Description
User Interface	React web UI (live pipeline, sources panel, report reader), CLI (app/cli.py) and Streamlit app
LLM	Google Gemini (gemini-3.5-flash-lite) used by the planner, synthesizer and reflector through llm.py
Agent	Compiled LangGraph StateGraph (graph.py) that owns the control flow
Planner	planner.py: question to 3-6 validated sub-queries
Reasoning Module	reflector.py (critique and follow-up queries) and route_after_reflection (decision)
Tools	tools/search.py (Tavily) and tools/scrape.py (httpx, trafilatura, BeautifulSoup)
Memory	ResearchState in state.py: short-term state with accumulating reducers
Knowledge Base	None pre-built; the live web is queried for every run
Retriever	searcher.py: runs the queries, de-duplicates URLs and scrapes pages in parallel
Executor	LangGraph runtime via stream_research(); the web server runs it in a worker thread and streams events
Output Generator	synthesizer.py writes the cited draft; finalizer.py validates citations and saves the report

4.3 Agent Roles and Responsibilities

Component Responsibilities

The system is a single agent. Table 7 maps the generic agent components to the modules that implement them.

Table 7: Component responsibilities

Component/Agent	Responsibility
Main Agent	The LangGraph graph: runs the nodes in order and applies the router's loop/finish decision
Tool Executor	Searcher node: calls web_search for each query and scrape for each selected URL
Retriever	search.py and scrape.py: return search hits and clean page text (or None)
Memory Module	ResearchState: keeps the plan, sources, drafts, critiques, iteration and token totals

Role-Specialised Nodes

Inside the single agent, the work is split across role-specialised nodes. They are not independent agents: they share one LLM client and one state object, and the graph, not the nodes, decides the order. Their roles are given in Table 8.

Table 8: Role-specialised nodes of the agent graph

Node	Role	Responsibility
Node 1	Planner	Decompose the question into 3-6 sub-queries (structured output); reset iteration to 0
Node 2	Searcher (researcher)	Search and scrape new sources for the sub-queries or follow-up queries; no LLM call

Node	Role	Responsibility
Node 3	Synthesizer (writer)	Write a cited Markdown draft from the numbered sources, addressing the last critique
Node 4	Reflector (reviewer)	Score the draft 1-10, list gaps and follow-up queries; increment iteration
Node 5	Finalizer	Remove invalid citations and save the report and first draft

4.4 Agent Workflow

The complete workflow of one run is as follows:

Step 1: The user submits a research question; the server checks that it is 1-500 characters, that the API keys are configured and that no other job is running.

Step 2: The planner receives the question and today's date and asks Gemini for a Plan object; the validator trims, de-duplicates and enforces 3-6 queries.

Step 3: The resulting sub-queries form the task plan; the iteration counter is set to 0.

Step 4: The searcher runs each query on Tavily and keeps the top 3 results whose URLs are not already in the state.

Step 5: The pages are scraped in parallel; pages that cannot be scraped fall back to the search engine's extract.

Step 6: The synthesizer receives all numbered sources (and the previous critique, if any) and writes a cited Markdown draft; the Sources list is appended in code.

Step 7: The reflector scores the draft from 1 to 10 and lists gaps and follow-up queries; pass/fail is derived in code from the score (≥ 8).

Step 8: The router sends the run back to Step 4 with the follow-up queries unless the draft passed, 3 rounds were reviewed, 400,000 tokens were used, or no follow-up queries were proposed.

Step 9: The finalizer removes invalid citations, saves the report (and the first draft if it was revised) and the UI displays the final report.

Figure 2 shows this workflow as implemented in `src/agent/graph.py`.

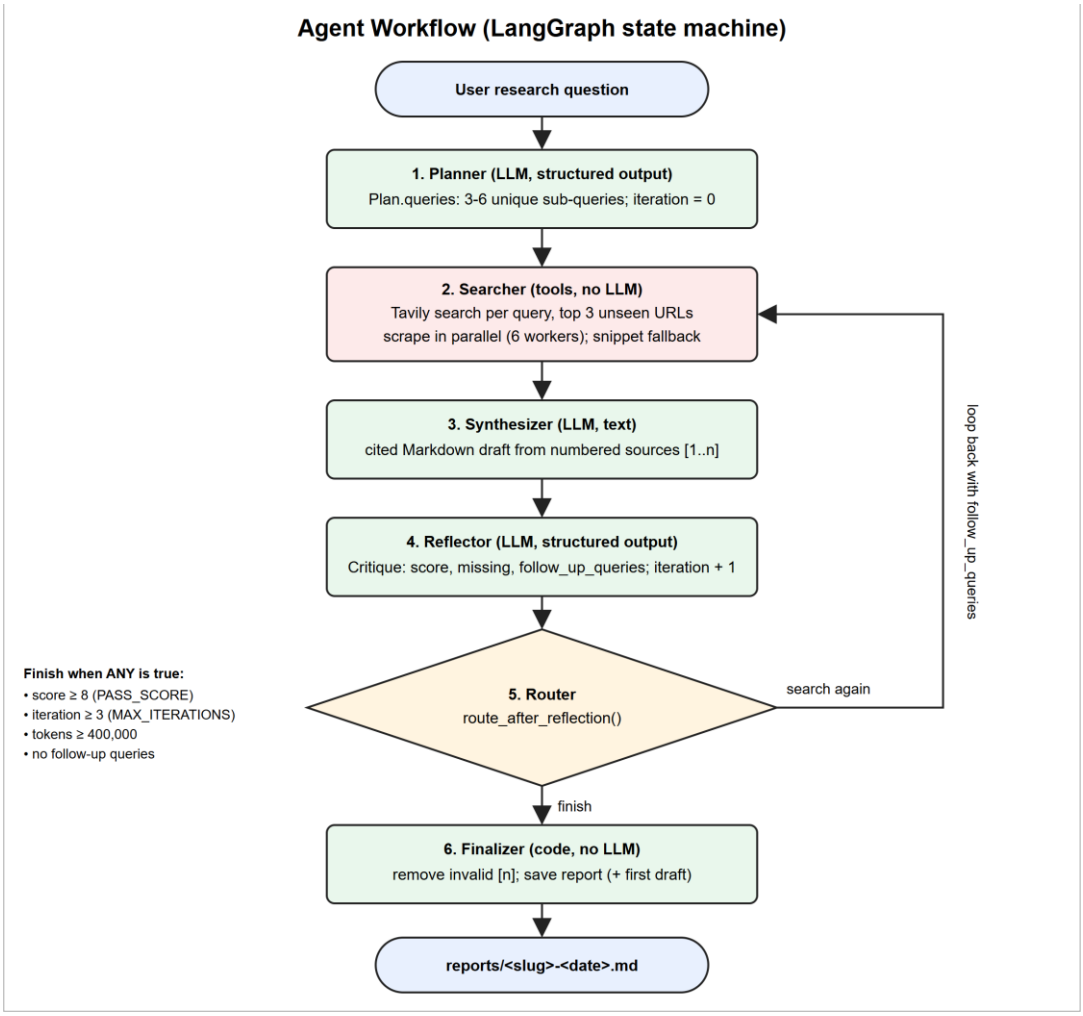


Figure 2: Agent Workflow

CHAPTER 5 – AGENTIC AI CONCEPTS USED

Only the concepts that are actually implemented are described as implemented. Table 9 summarises the status of each concept listed in the template; the subsections give the details.

Table 9: Agentic AI concepts and their implementation status

Concept	Status in this project	Where
ReAct	Not used as a prompting technique; a fixed plan-act-reflect loop is used instead	graph.py
Planning	Implemented (task decomposition, plan-and-execute, re-planning from critique)	planner.py, reflector.py
Tool calling	Implemented (web search and scraping, invoked by the graph)	tools/, searcher.py
Memory	Short-term working memory only	state.py
RAG	Web retrieval-augmented generation; no embeddings or vector database	searcher.py, synthesizer.py
Multi-agent system	Not implemented; one agent with role-specialised nodes	nodes/
Human-in-the-loop	No approval step; the user starts and can stop a run	web.py
Self-reflection	Implemented (critique, score, follow-up searches)	reflector.py

5.1 ReAct (Reason → Act → Observe)

ReAct [3] interleaves free-form reasoning traces with tool actions inside one prompt, and the model chooses each next action. **This project does not use ReAct prompting.** Instead, the Reason → Act → Observe idea is realised at the level of the graph:

- **Reason:** the reflector reads the draft (the observation of the previous actions) and reasons about missing coverage, weak support and conflicts.
- **Act:** its follow-up queries are executed by the searcher in the next round.
- **Observe:** the new sources flow into a new draft, which is reviewed again.

The order of steps is fixed by the graph, and the LLM decides only what to search for and whether the draft is good enough. This makes the loop predictable and testable.

5.2 Planning

- **Task decomposition:** the planner splits the question into 3-6 specific, non-overlapping search queries that together cover it.

- **Plan-and-Execute:** the plan is produced once, before any tool is used, and the searcher then executes every planned query.
- **Goal-based planning:** the goal is an explicit, checkable condition: a review score of 8/10, under limits of 3 rounds and 400,000 tokens.
- **Dynamic planning:** after each review the reflector's follow-up queries act as a revised plan for the next search round, so the agent re-plans around detected gaps.

5.3 Tool Calling

- **Tools used:** a Tavily web search wrapper (`web_search`) and a web page scraper (`scrape`). The finalizer also writes files to disk.
- **Purpose:** search finds relevant pages for a query; the scraper extracts the main text of each page as evidence.
- **Tool selection mechanism:** tools are not chosen by LLM function calling. The graph always routes to the searcher, which calls search and then scrape. The LLM influences tool use through the queries it writes (planner and reflector).
- **Tool input:** a query string for search; a URL for the scraper.
- **Tool output:** a list of {title, url, snippet} hits; clean page text of up to 6,000 characters, or None on failure.
- **How the output is used:** each source is appended to the state with its content and a scraped flag, numbered, and given to the synthesizer as the only allowed evidence.

Table 10 summarises the tools.

Table 10: Tools used by the agent

Tool	Purpose	Input	Output
web_search (Tavily)	Find relevant pages for a query	Query string, max_results = 5	List of {title, url, snippet}
scrape	Extract main text from a page	URL	Up to 6,000 characters of text, or None
finalizer file writer	Persist the result	Final draft, first draft	reports/<slug>-<date>.md (+ -draft1.md)

5.4 Memory

- **Short-term memory (implemented):** the ResearchState TypedDict holds the question, plan, sources, drafts, critiques, iteration count and token count for the duration of one run. Fields annotated with `operator.add` accumulate across loops, so sources gathered in round 1 are still available in round 3 and a citation [4] refers to the same page in every draft.
- **Long-term memory:** not implemented. Reports and run metadata are saved to disk and listed in the UI library, but the agent never reads them back in later runs.
- **Conversational memory:** not implemented; each run answers one question.
- **Vector-based memory:** not implemented; no embeddings or vector store are used.
- **Memory retrieval:** nodes read previous results directly from the state, for example the synthesizer reads the last critique and the searcher reads the set of URLs already seen.

5.5 Retrieval-Augmented Generation

The agent uses web-based retrieval-augmented generation. Compared with the classic pipeline Documents → Chunking → Embeddings → Vector Database → Retrieval → Context → LLM → Response [2], the implemented pipeline is:

Query → Web search (Tavily) → Scrape and clean page → Truncate (6,000 chars) → Number sources → Context → LLM (Gemini) → Cited response

- **Documents:** live web pages returned by the search API for each sub-query.
- **Chunking:** not used; each page is cleaned and truncated to its first 6,000 characters.
- **Embeddings and vector database:** not used; ranking is done by the search engine.
- **Retrieval:** the top 3 unseen results per query, scraped in parallel.
- **Context:** all sources are formatted as numbered blocks [1]..[n] with title and URL.
- **LLM and response:** Gemini writes the report citing only those numbers; the reference list is generated from the state, not by the model.

5.6 Multi-Agent System

Not implemented. The system is a single agent. Its five nodes have distinct roles (Table 8), but they do not communicate with each other or delegate tasks; they share one state object and are orchestrated by one graph with a fixed order and one conditional edge. Turning the roles into independent collaborating agents is listed as future work.

5.7 Human-in-the-Loop

No approval step is implemented. A human provides the question and can press Stop (or close the browser tab) at any time; the server then stops the graph after the current step. After the run, the user reviews the report, the sources and the first draft. Adding approval of the research plan before searching is listed as future work.

CHAPTER 6 – METHODOLOGY

6.1 Input Processing

The question enters through POST /api/research (web UI), the command-line arguments (CLI) or a text box (Streamlit). The web server rejects bodies that are not JSON objects and questions that are empty or longer than 500 characters (HTTP 400), returns HTTP 503 if the API keys are missing and HTTP 409 if a job is already running. The trimmed question becomes the initial state {"question": ...}.

6.2 Task Understanding

The system does not use a separate intent classifier: every input is treated as a research question. Understanding happens in the planner prompt, which gives Gemini today's date and asks for queries that together cover everything needed to answer the question fully, adding a recency term for time-sensitive topics.

6.3 Task Planning

The planner calls `structured_call(Plan, prompt)`. Gemini returns JSON matching the Plan schema; the validator strips whitespace, removes blanks and case-insensitive duplicates, rejects fewer than 3 queries and keeps at most 6. If the JSON does not parse, the call is repeated once with the validation error appended to the prompt.

6.4 Reasoning and Decision Making

Reasoning about quality happens in the reflector, which checks coverage of each sub-query, citation support, accuracy and depth, and returns a Critique (score, passed, missing, follow_up_queries). The decision of what to do next is made in code by `route_after_reflection`: finish if the score is at least 8 (the model's own passed flag is overridden by the score), if 3 rounds were reviewed, if 400,000 tokens were used, or if no follow-up queries were proposed; otherwise search again.

6.5 Tool Selection

Tool selection is deterministic. The searcher always uses the search tool for each query and the scraper for each selected URL. Round 1 uses the planner's sub-queries; later rounds use the reflector's follow-up queries. URLs already in the state are skipped so no page is added twice.

6.6 Task Execution

LangGraph executes the nodes and passes the state between them. Searches run sequentially with up to 3 attempts and exponential back-off; scraping runs in a thread pool of 6 workers with a 10 s timeout, a check of robots.txt and at least 1 s between requests to the same host. In the web UI the graph runs in a worker thread so that the server stays responsive.

6.7 Observation and Feedback

Each node's output is merged into the state through reducers. The synthesizer observes all sources and the previous critique; the reflector observes the draft and the plan; the router observes the critique, iteration and token counts. In parallel, every node update is streamed to the user interface as a numbered event, so the user sees the same observations.

6.8 Final Response Generation

When the router chooses to finish, the finalizer validates every [n] citation against the number of sources, removes invalid numbers, and saves the report as reports/<slug>-<date>.md, plus the first draft (-draft1.md) when the loop revised it. The web server also writes run metadata (.json). The UI then opens the report in a reader with citation previews, a table of contents, Markdown download, print-to-PDF and draft comparison.

CHAPTER 7 – IMPLEMENTATION

7.1 Project Structure

The actual folder structure of the repository is shown below (generated files such as app/static/assets are summarised).

```
Autonomous-Deep-Research-AI-Agent/
├── src/agent/
│   ├── config.py          # environment, model name, limits
│   ├── state.py           # ResearchState + Plan / Critique schemas
│   ├── llm.py             # Gemini client, rate limiting, text/structured calls
│   ├── prompts.py         # planner, synthesizer and reflector prompts
│   ├── graph.py           # StateGraph wiring and router
│   ├── nodes/
│   │   ├── planner.py  searcher.py  synthesizer.py
│   │   └── reflector.py  finalizer.py
│   └── tools/
│       ├── search.py      # Tavily wrapper
│       └── scrape.py      # httpx + trafilatura scraper
├── app/
│   ├── web.py             # Starlette server + streaming API
│   ├── cli.py  run.py  streamlit_app.py  evaluate.py
│   ├── frontend/          # React + TypeScript source (Vite)
│   └── static/            # built web UI (generated)
├── tests/                 # 7 pytest files (unit + end-to-end)
├── docs/                  # report, screenshots
├── reports/               # generated reports (git-ignored)
├── requirements.txt  pytest.ini  .env.example
└── README.md
```

7.2 Agent Implementation

The agent is a LangGraph StateGraph. The snippet below from `src/agent/graph.py` wires the five nodes and the conditional edge; this is the whole control flow of the agent.

```
def route_after_reflection(state: ResearchState) -> str:
    if state["critique"]["passed"] or state["iteration"] >= config.MAX_ITERATIONS:
        return "finalizer"
    if state.get("tokens_used", 0) >= config.MAX_TOKENS_PER_RUN:
        return "finalizer"
    if not state.get("follow_up_queries"):
        return "finalizer"  # nothing new to search for
    return "searcher"


g = StateGraph(ResearchState)
g.add_node("planner", planner)      # ... searcher, synthesizer, reflector, finalizer
g.set_entry_point("planner")
g.add_edge("planner", "searcher")
g.add_edge("searcher", "synthesizer")
g.add_edge("synthesizer", "reflector")
g.add_conditional_edges("reflector", route_after_reflection,
                        {"searcher": "searcher", "finalizer": "finalizer"})
g.add_edge("finalizer", END)
```

Explanation: the router is plain Python, so every stopping rule can be unit tested (tests/test_router.py). The reflector increments `iteration` before the router runs, so the cap counts drafts already reviewed.

LLM outputs that drive control flow use Pydantic structured output. The Critique schema and the reflector's override of the pass flag are shown below.

```
class Critique(BaseModel):
    score: int = Field(description="Overall quality from 1 (poor) to 10 (excellent)")
    passed: bool = Field(description="True only if score >= 8")
    missing: list[str]
    follow_up_queries: list[str]


# in reflector(): the score decides, not the model's own passed flag
critique.passed = critique.score >= config.PASS_SCORE
critique.follow_up_queries = critique.follow_up_queries[: config.MAX_FOLLOW_UP_QUERIES]
```

Explanation: `structured_call` in `llm.py` calls Gemini with `with_structured_output(schema, method="json_schema")`; if parsing fails it retries once with the error message. Deriving `passed` from the score in code prevents an inconsistent model answer (for example score 6 with passed = true) from ending the loop.

The LLM wrapper also protects the free-tier quota: an in-memory rate limiter spaces calls to 4 per minute, a counter stored in .gemini_usage.json enforces a daily cap, and on HTTP 429 the call waits for the delay suggested by the server before retrying.

7.3 Tool Implementation

Tool Name: web_search (src/agent/tools/search.py)

- **Purpose:** find relevant web pages for one query.

- **Input:** query string; max_results (default 5, i.e. 3 needed + 2 spare).
- **Output:** list of {title, url, snippet}.
- **Implementation:** TavilyClient.search with up to 3 attempts and exponential back-off (1 s, 2 s); raises a clear error after the last attempt, which the searcher logs and skips.
- **Role in Agent Workflow:** first action of every search round.

Tool Name: scrape (src/agent/tools/scrape.py)

- **Purpose:** extract the main readable text of a page.
- **Input:** URL.
- **Output:** up to 6,000 characters of clean text, or None.
- **Implementation:** see the snippet below; any exception returns None so a bad page can never crash the graph.
- **Role in Agent Workflow:** turns search hits into evidence for the synthesizer.

```
def scrape(url: str) -> str | None:
    try:
        if urlparse(url).scheme not in ("http", "https") or not _allowed(url):
            return None                       # robots.txt disallows it
        _wait_for_host(urlparse(url).netloc)  # >= 1 s between hits to one host
        response = _get(url)                  # httpx, 10 s timeout, browser User-Agent
        if response.status_code != 200:
            return None
        text = _extract(response.text)        # trafilatura, BeautifulSoup fallback
        if len(text) < config.MIN_CONTENT_CHARS:
            return None
        return text[: config.MAX_CONTENT_CHARS]
    except Exception:
        return None
```

In the searcher, a page that returns None is not discarded: `content = text or hit.get("snippet", "")` keeps the search engine's extract, and the source is marked `scraped: False` so the UI can flag it as a fallback.

7.4 Memory Implementation

- **Memory type:** short-term working memory for one run.
- **Storage mechanism:** a LangGraph state dictionary (ResearchState, a TypedDict) held in memory.
- **Data stored:** question, sub-queries, sources, current draft and all drafts, current critique and all critiques, follow-up queries, iteration, tokens used, final report and path.

- **Retrieval mechanism:** nodes read fields directly from the state passed to them.
- **How memory influences decisions:** the searcher skips URLs already seen; the synthesizer addresses the previous critique; the router uses the iteration and token totals.

```
class ResearchState(TypedDict, total=False):
    question: str
    sub_queries: list[str]
    sources: Annotated[list[dict], operator.add]   # appended, never replaced
    draft: str
    drafts: Annotated[list[str], operator.add]
    critique: dict
    critiques: Annotated[list[dict], operator.add]
    follow_up_queries: list[str]
    iteration: int
    tokens_used: Annotated[int, operator.add]
    final_report: str
```

Long-term, conversational and vector memory are not implemented (see Section 5.4).

7.5 RAG Implementation

- **Data collection:** Tavily search results for each planned or follow-up query.
- **Document loading:** httpx GET of each URL (robots.txt checked, 10 s timeout).
- **Document preprocessing:** trafilatura main-text extraction; if it yields under 200 characters, BeautifulSoup text with scripts, navigation and footers removed.
- **Chunking:** not used; text is truncated to 6,000 characters per page.
- **Embedding generation and vector database:** not used.
- **Similarity search:** delegated to the search engine's ranking; the agent keeps the top 3 new URLs per query.
- **Context retrieval:** all sources in the state are formatted as numbered blocks.
- **LLM response generation:** the synthesizer prompt allows only the numbered sources as evidence and requires a citation on every factual claim.

```
def format_sources(sources: list[dict]) -> str:
    return "\n\n".join(
        f"[{i}] {s['title']}\nURL: {s['url']}\n{s['content']}"
        for i, s in enumerate(sources, start=1))


def sources_section(sources: list[dict]) -> str:
    """Build the Sources list from state, so URLs are never model-written."""
    lines = [f"{i}. [{s['title']}]({s['url']})" for i, s in enumerate(sources, start=1)]
    return "## Sources\n\n" + "\n".join(lines)
```

The finalizer's `validate_citations` then removes any [n] greater than the number of sources, so a report can only cite pages that were actually retrieved.

7.6 User Interface

The web interface was captured during a real run on the question "How does retrieval-augmented generation work, and what are its main failure modes?". Figures 3 to 8 show the main screens.

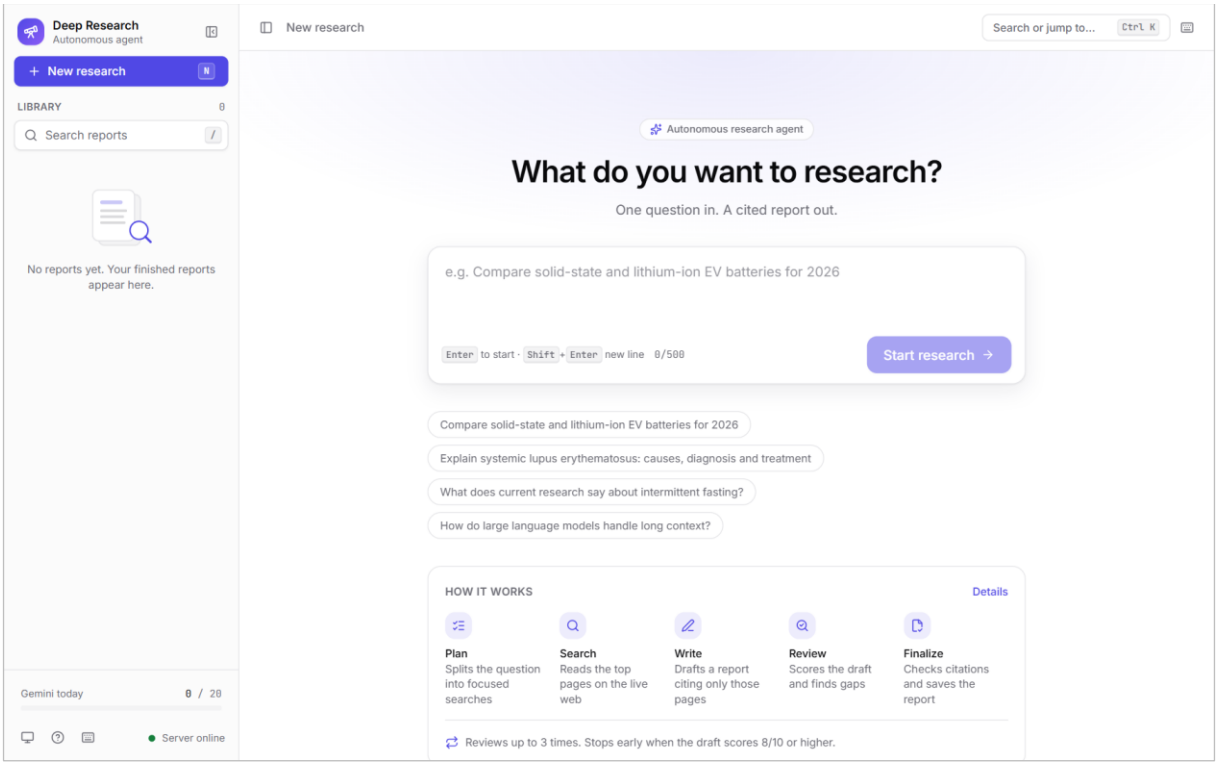


Figure 3: Home page of the web interface

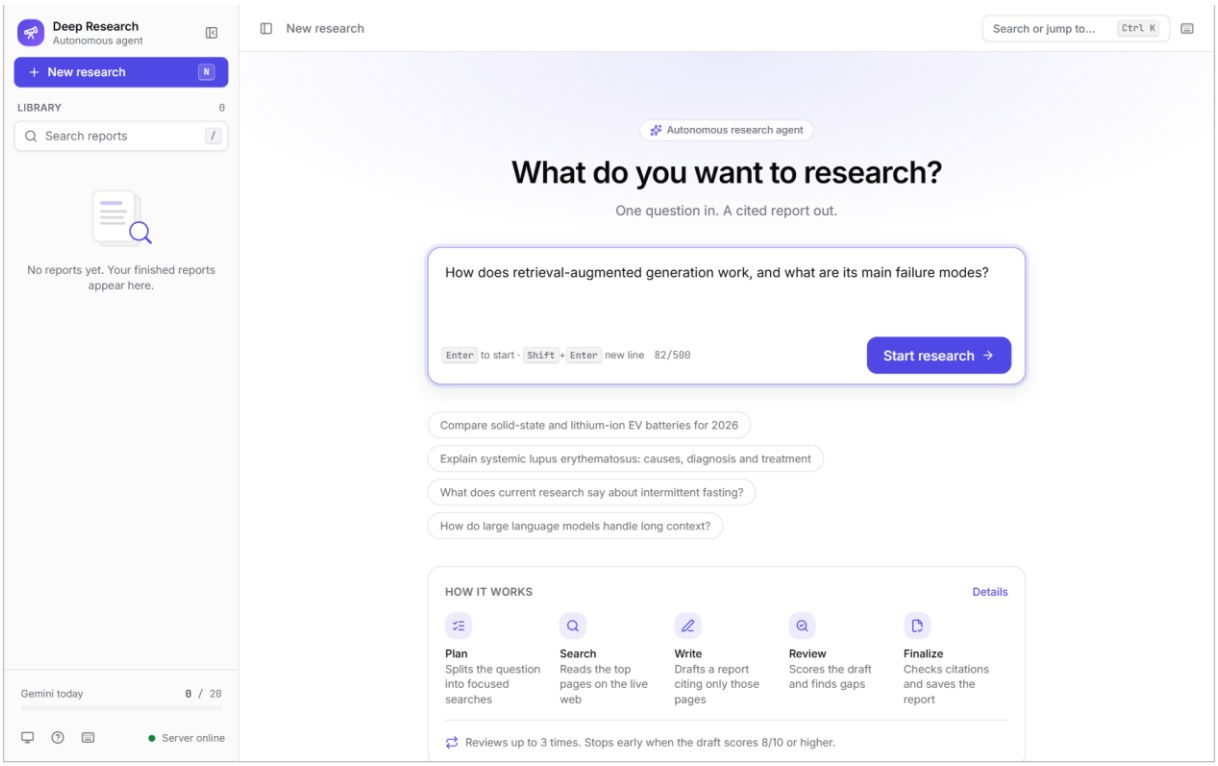


Figure 4: Input interface with a research question entered

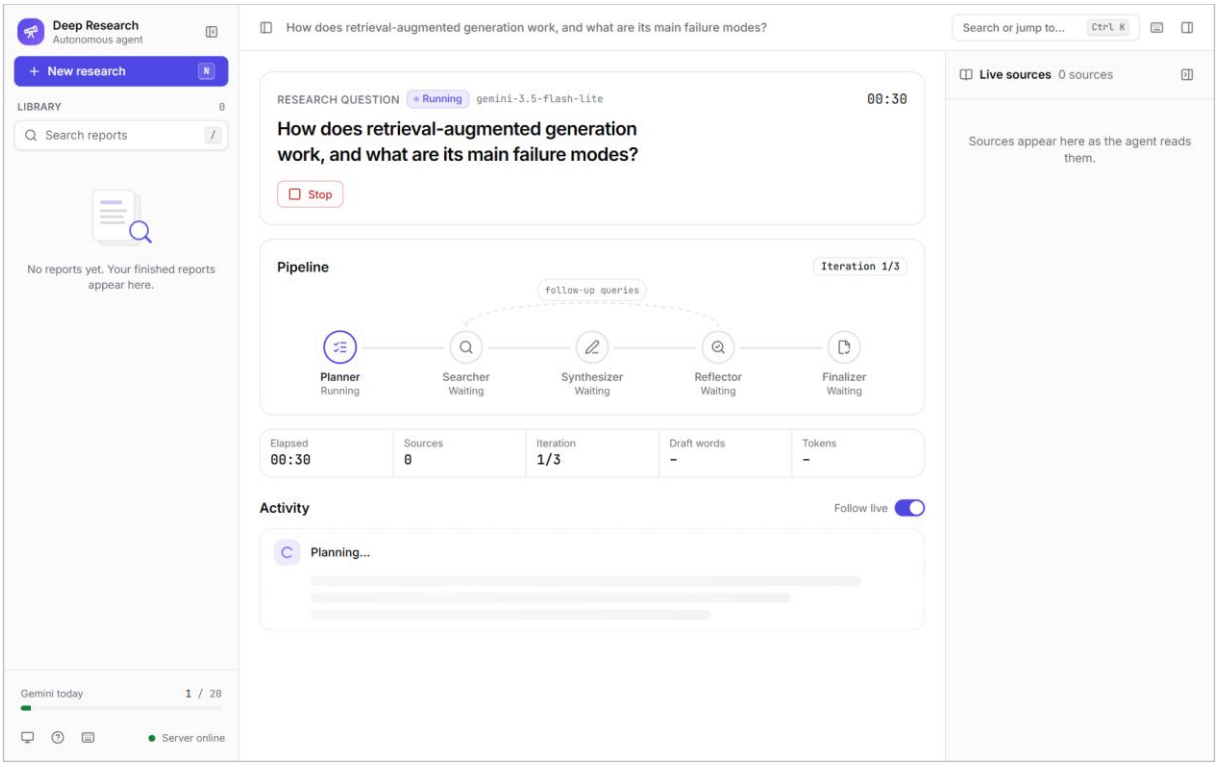


Figure 5: Agent execution: run started, planner decomposing the question

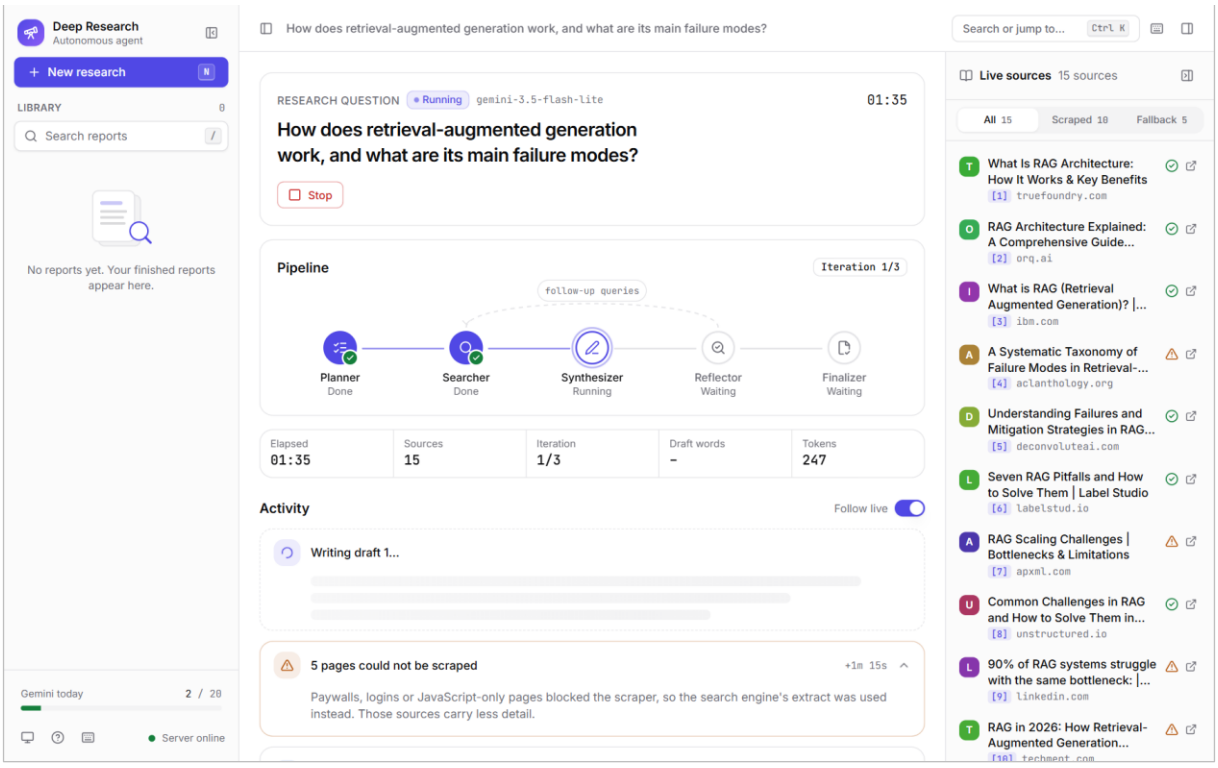


Figure 6: Tool execution: live sources retrieved and scraped by the searcher

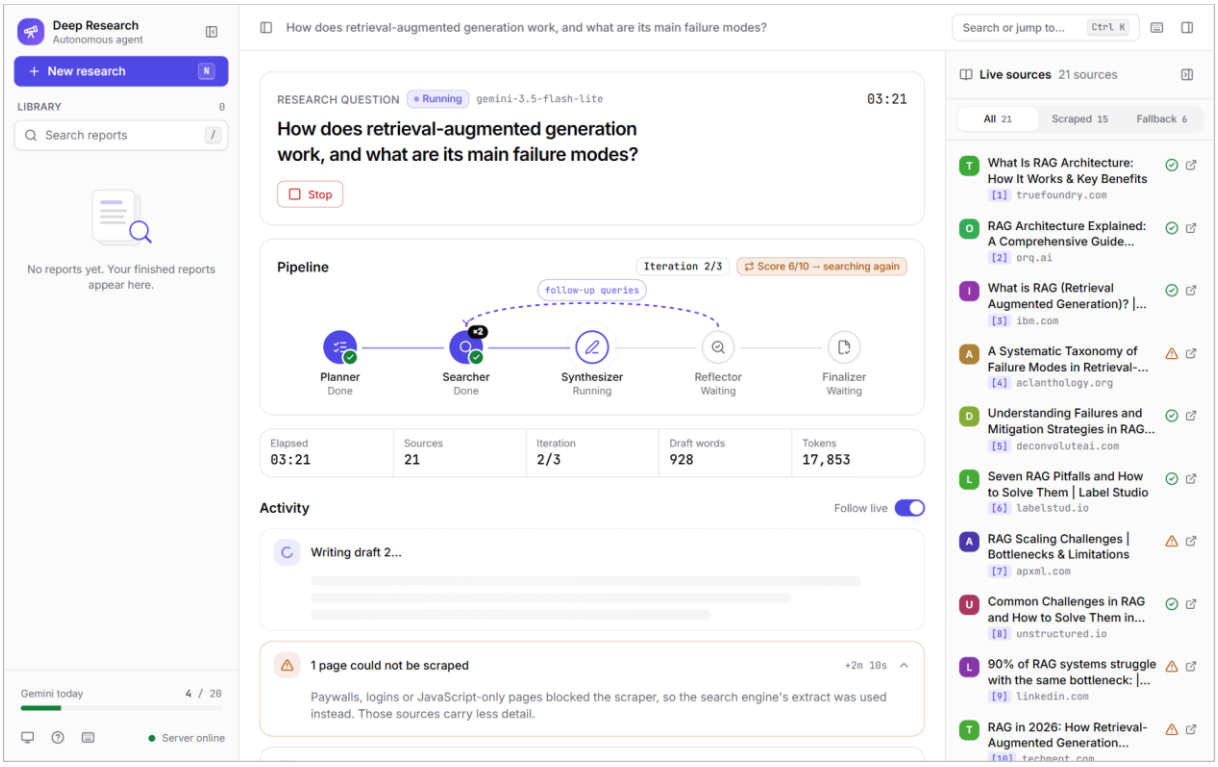


Figure 7: Reflection loop: draft scored 6/10, agent searching again

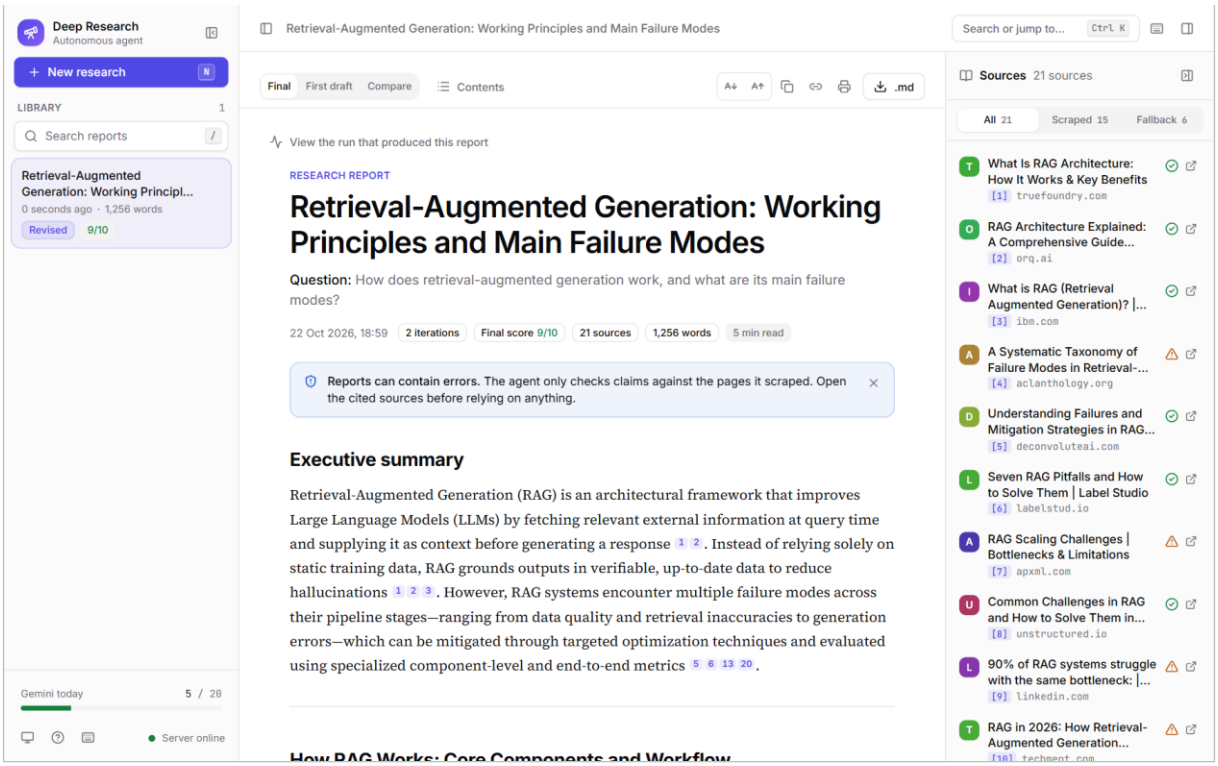


Figure 8: Final output: finished report with score, iterations and sources

CHAPTER 8 – RESULTS AND OUTPUT

8.1 Test Cases

The system was tested with automated tests and live runs. The backend suite has 62 pytest tests, of which 2 live-API tests are deselected by default; the 60 offline tests use fake search, scrape and LLM functions and all passed in 3.32s. The frontend suite has 44 Vitest tests in 6 files, all of which passed. Table 11 lists representative test cases with their actual results; TC01 and TC02 are live runs.

Table 11: Test cases

Test Case ID	Input	Expected Output	Actual Output	Status
TC01	Live web UI run: "How does retrieval-augmented generation work, and what are its main failure modes?"	Cited report saved within 3 review rounds	Report saved after 2 rounds (scores 6 → 9), 21 sources, 306 s	Pass
TC02	Live CLI runs: EV batteries; nuclear power	Cited reports saved; loop stops early when score ≥ 8	Both passed on review 1 (score 9 and 9); no extra rounds	Pass
TC03	POST /api/research with {}, blank, 501-char question, or a JSON list	HTTP 400, no job started	HTTP 400 for all 4 inputs	Pass
TC04	Second question while a job is running	HTTP 409 with the running job's id	HTTP 409 with run_id	Pass
TC05	Fake LLM scores draft 5 then 9	One follow-up search round, then finish; first draft saved	iteration = 2, scores [5, 9], 4 searches, report + draft1 saved	Pass
TC06	Fake LLM always scores 1	Stop at MAX_ITERATIONS (3)	iteration = 3, finalizer reached	Pass
TC07	Search returns no results	Run still finishes with an "insufficient evidence" report	Report saved	Pass
TC08	Report text with [2, 7] and [9] but 4 sources	Invalid numbers removed	"B [2]. C ."; removed [7, 9]	Pass
TC09	Scrape: timeout, connection error, 404, empty, PDF, robots.txt disallow	Return None, never raise	None for all cases	Pass
TC10	GET /api/reports/..%2F..%2F.env	HTTP 404 (no path traversal)	HTTP 404	Pass

Figure 9 shows the actual console output of both test suites.

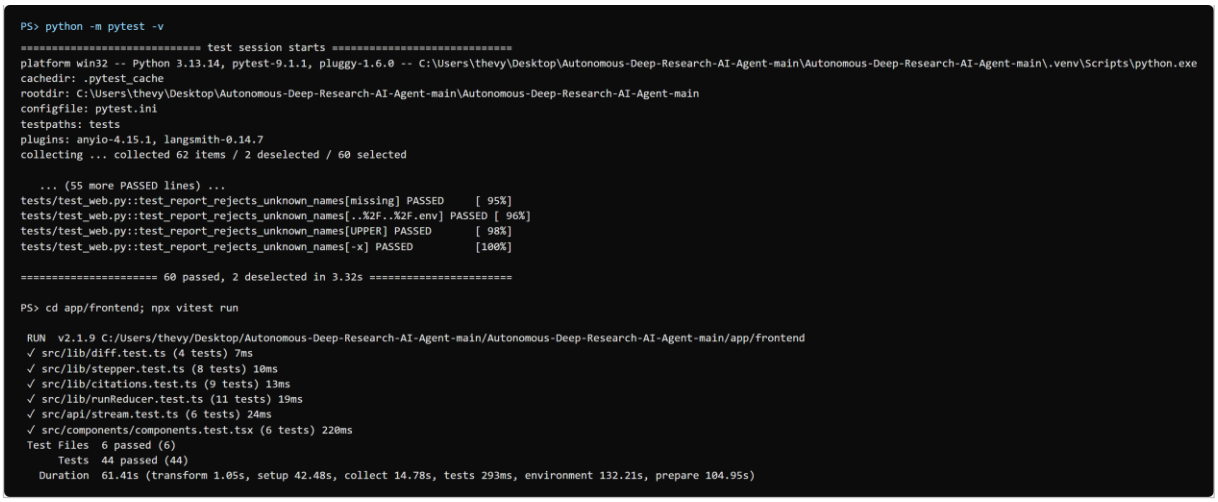


Figure 9: Automated test results (pytest and Vitest)

8.2 Sample Outputs

This section shows each live run as input, agent action, output and screenshot. Table 12 summarises all live runs.

Table 12: Results of the live research runs

Run	Question	Interface	Rounds	Scores	Sources (scraped)	Time (s)	Tokens
1	How does retrieval-augmented generation work, and what are its main failure modes?	Web UI	2	6 → 9	21 (15)	306	43,270
2	Compare solid-state and lithium-ion EV batteries for 2026	CLI	1	9	15 (12)	127	20,501
3	Is nuclear power necessary to reach net-zero emissions by 2050?	CLI	1	9	15 (5)	186	12,410

Test Case 1: Web UI run (technical question)

Input: "How does retrieval-augmented generation work, and what are its main failure modes?"

Agent Action: planned 5 sub-queries; retrieved 15 sources (10 scraped); wrote a 928-word draft scored 6/10 with gaps on bottlenecks, improvement techniques and evaluation metrics; searched 3 follow-up queries (6 new sources); rewrote the report (1256 words), which scored 9/10 and passed.

Output: a cited report titled "Retrieval-Augmented Generation: Working Principles and Main Failure Modes" with 21 sources, saved after 306 s and 43,270 tokens.

Screenshot: Figure 10 and Figure 11.

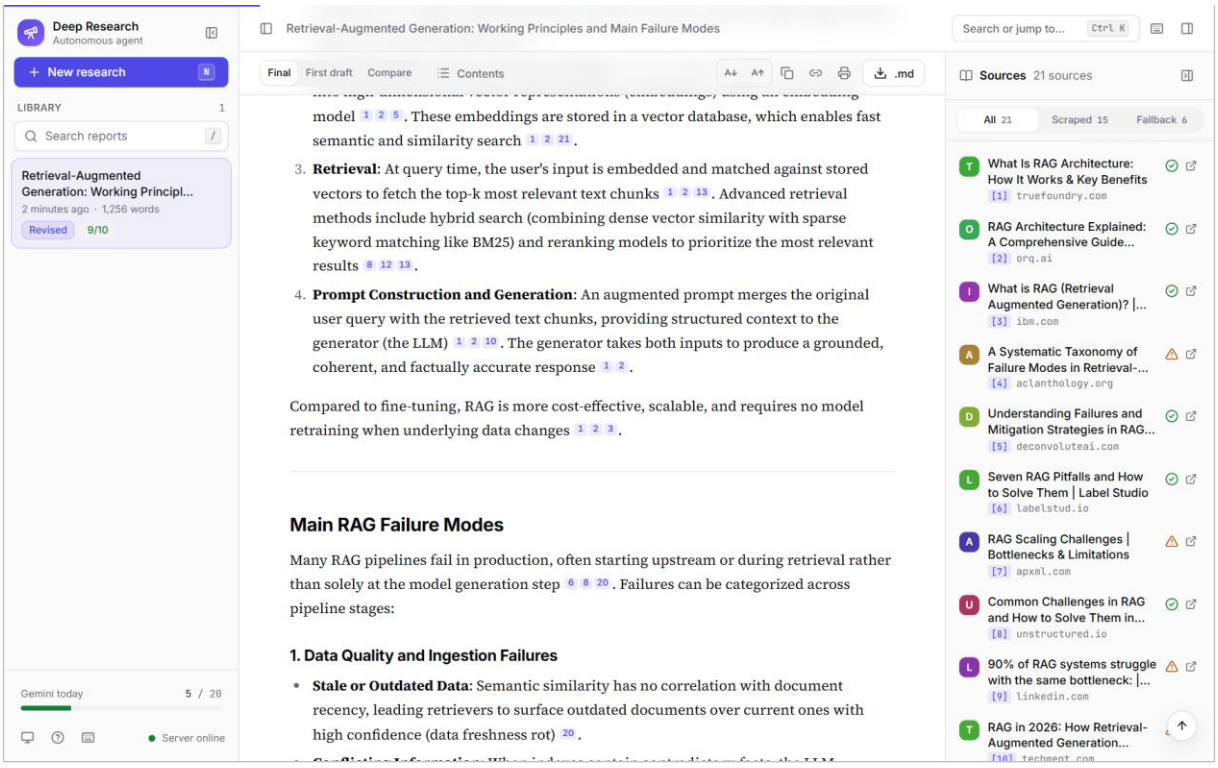


Figure 10: Sample output: report body with inline citations

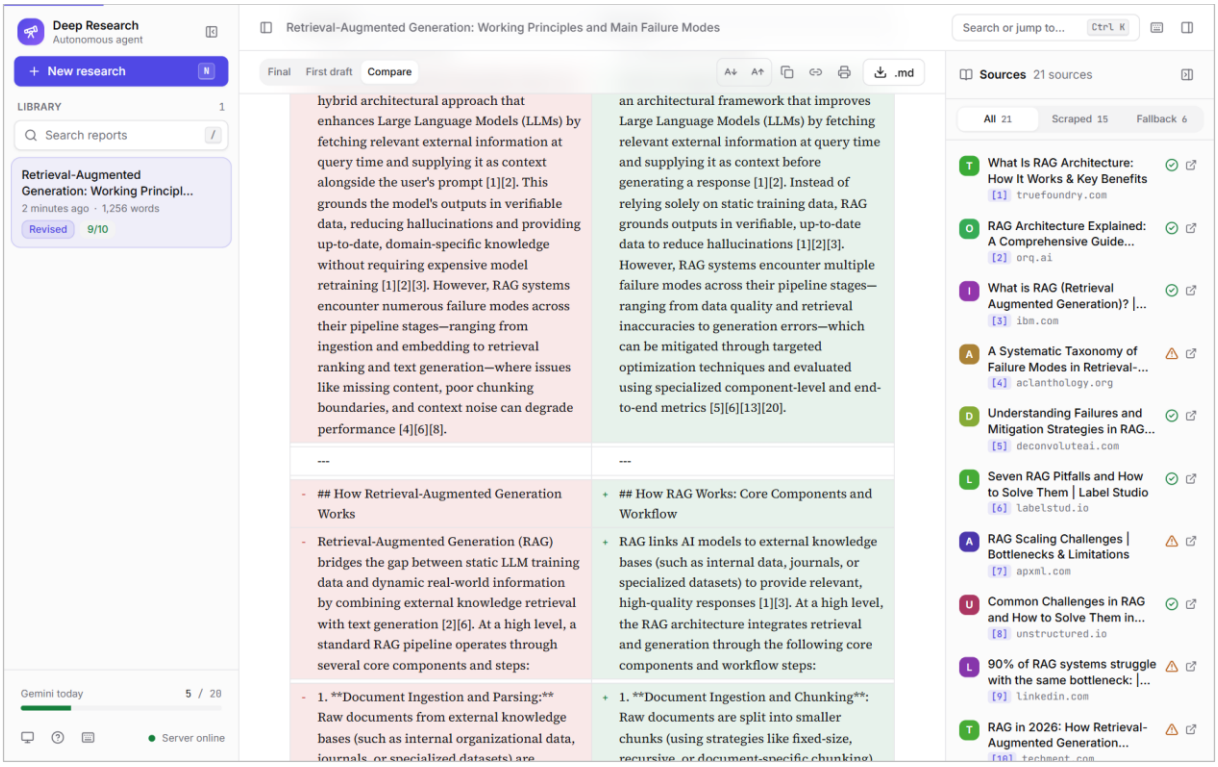


Figure 11: Sample output: first draft compared with the revised report

Test Case 2: CLI run (EV battery comparison)

Input: `python -m app.cli "Compare solid-state and lithium-ion EV batteries for 2026"`

Agent Action: planned 5 sub-queries; retrieved 15 sources (12 scraped, 3 snippet fallback); wrote a 933-word draft; the reflector scored it 9/10, so the router finished without a second round.

Output: completed in 1 review round(s), scores 9, 15 sources, 127 s, 20,501 tokens.

Screenshot: Figure 12.

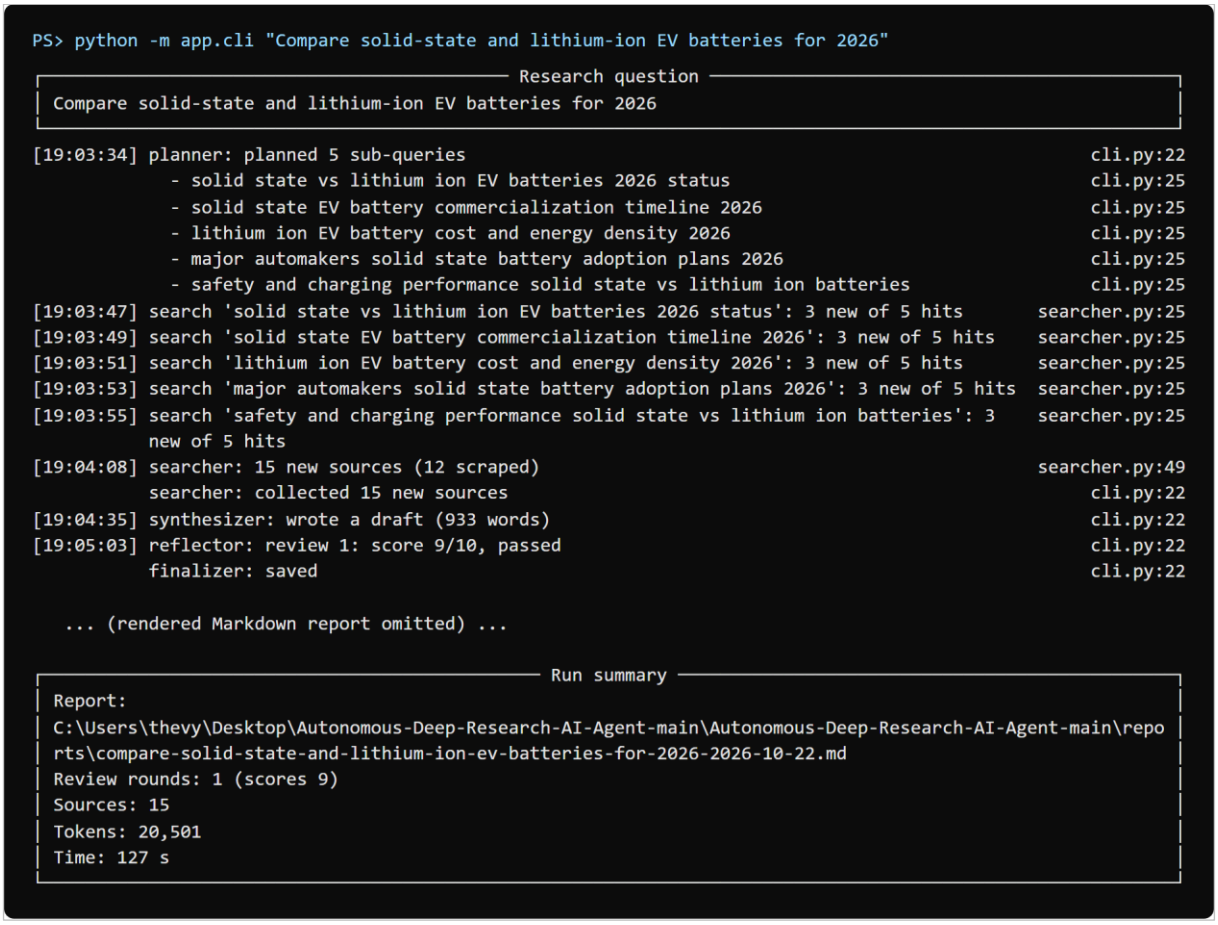


Figure 12: CLI run: EV battery comparison

Test Case 3: CLI run (nuclear power and net zero)

Input: `python -m app.cli "Is nuclear power necessary to reach net-zero emissions by 2050?"`

Agent Action: planned 5 sub-queries; retrieved 15 sources (5 scraped, 10 snippet fallback); wrote a 734-word draft; the reflector scored it 9/10, so the router finished without a second round.

Output: completed in 1 review round(s), scores 9, 15 sources, 186 s, 12,410 tokens.

Screenshot: Figure 13.

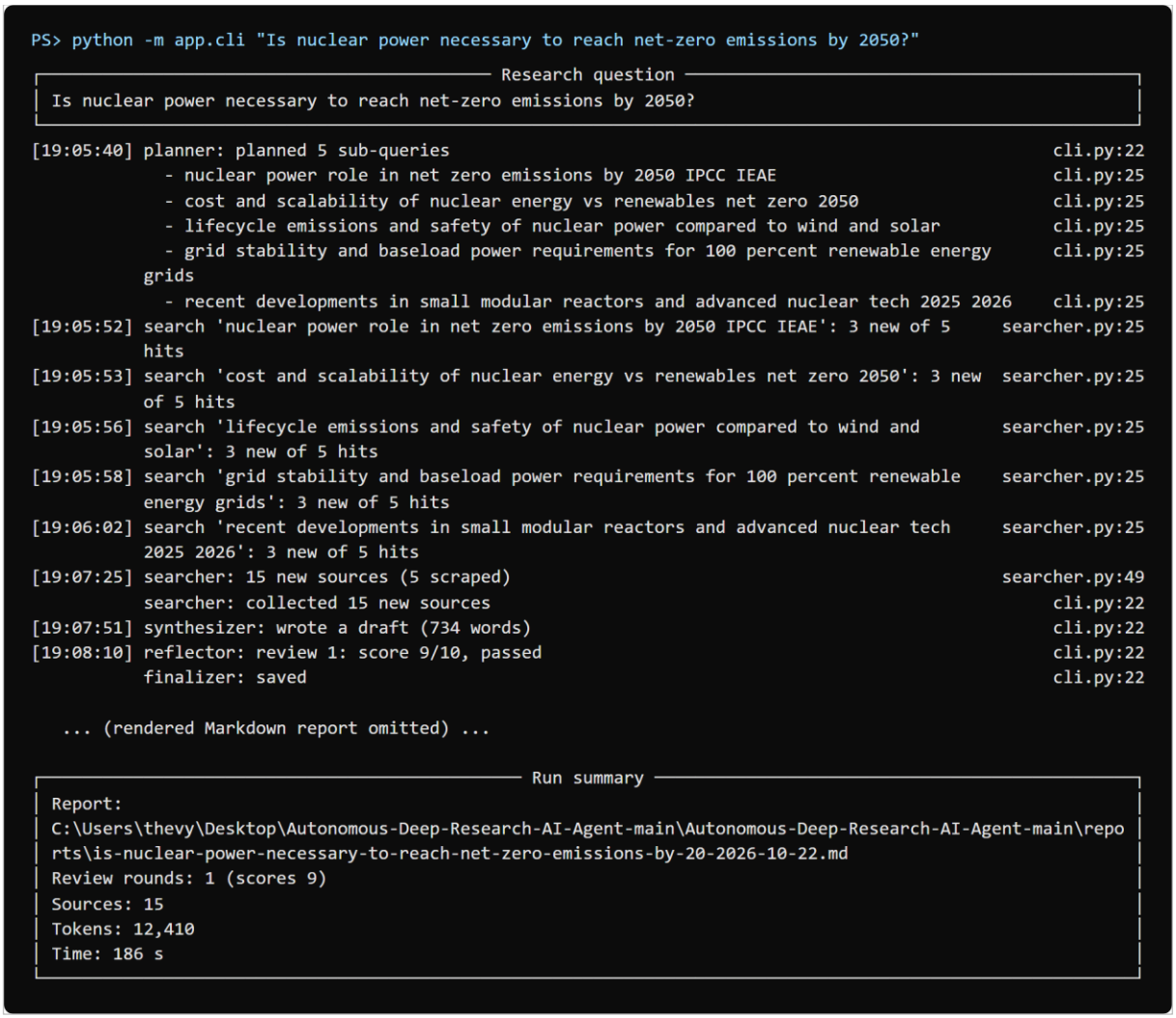


Figure 13: CLI run: nuclear power and net zero

8.3 Agent Execution Trace

The execution sequence below is the actual event log of live run 1 (web UI), recorded from the server's event stream.

User Query: "How does retrieval-augmented generation work, and what are its main failure modes?"

↓

Task Understanding: question validated (1-500 chars) and placed in the initial state

↓

Plan Generation: planner produced 5 sub-queries, e.g. "main failure modes of retrieval augmented generation systems"

↓

Tool Selection: searcher ran Tavily for each sub-query and kept the top 3 new URLs

↓

Execution: 15 sources collected, 10 scraped, 5 used the snippet fallback; draft 1 written (928 words)

↓

Observation: reflector scored draft 1 at 6/10 and listed 2 gaps

↓

Decision: router chose "searcher" (score below 8, iteration 1 of 3) with 3 follow-up queries

↓

Execution: 6 new sources (5 scraped); draft 2 written (1256 words)

↓

Observation: reflector scored draft 2 at 9/10 (passed)

↓

Decision: router chose "finalizer" (reason: passed)

↓

Final Response: report saved and shown in the UI after 306.4 s, 43,270 tokens

CHAPTER 9 – PERFORMANCE EVALUATION

Table 13 reports metrics measured on the live runs of Section 8.2 and on the automated test suites. All values are actual measurements; metrics that could not be measured objectively are marked as such.

Table 13: Performance evaluation

Metric	Evaluation Method	Result
Task Completion Rate	Completed runs / live runs started	3 / 3 (100%)
Citation Validity (accuracy proxy)	Citation numbers in final reports that match a listed source / all citation numbers	180 / 180 (100%); finalizer removed 1 invalid citation in run 1 ([2026])
Tool Success Rate (search)	Successful Tavily calls / total calls (from run logs)	18 / 18 (100%)
Tool Success Rate (scrape)	Pages scraped / pages selected	32 / 51 (62.7%); the rest used the snippet fallback
Average Response Time	Mean wall-clock time per completed run	206 s (runs: 306, 127, 186 s)
Average Token Usage	Mean input + output tokens per run	25,394 (runs: 43,270, 20,501, 12,410)
Self-assessed Quality	Reflector score of the final draft (1-10)	9, 9, 9 (all ≥ 8, passed)
Reflection Loop Effect	Score before and after a follow-up round	Run 1: 6 → 9 after 6 new sources
Retrieval Accuracy	Relevant retrievals / total retrievals	Not measured (no labelled relevance data)
Test Case Success Rate	Passed tests / total tests	104 / 104 (100%): 60 pytest + 44 Vitest

Notes on the measurements. The quality scores are produced by the agent's own reflector (the same Gemini model), so they are a self-assessment, not an independent measure of factual accuracy. Factual accuracy of individual claims and retrieval relevance were not measured because no labelled ground-truth dataset was available; citation validity is measured instead. Runs were executed on 22 October 2026 on the test machine described in Table 3 with the free-tier rate limit of 4 Gemini requests per minute, which accounts for a large share of the run time.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

The following challenges were met during development and testing:

- **API rate limits and cost:** the Gemini free tier allows only a few requests per minute and per day. We added a client-side rate limiter (4/min), a persisted daily cap and waiting on HTTP 429 errors; as a result most of a run's time is spent waiting for quota.
- **Scraping failures:** paywalled, login-only and JavaScript-rendered pages cannot be scraped. Across the live runs 32 of 51 pages (63%) were scraped; the rest fell back to the shorter search snippet.
- **LLM hallucination:** the model may still state something a source does not say. We reduce this by allowing only numbered sources, generating the Sources list in code and removing invalid citation numbers, but a wrong source still gives a wrong, correctly cited claim.
- **Inconsistent agent decisions:** in live run 1 the reflector's follow-up queries repeated three of the original sub-queries even though the prompt asks it not to; the URL de-duplication meant only new pages were added. The model could also return passed = true with a low score, which is why pass/fail is derived from the score in code.
- **Structured output errors:** occasional invalid JSON is handled by one retry with the validation error.
- **Context limitations and latency:** every source is truncated to 6,000 characters, and later rounds send all accumulated sources to the model, so token use grows each round (live run 1 used 17,853 tokens up to the first review and 43,270 in total).
- **Self-evaluation bias:** the same model writes and grades the draft, so a passing score does not guarantee correctness.
- **Dependency on external services:** the agent needs Gemini, Tavily and the target websites to be available; outages or quota exhaustion stop a run with a clear error.
- **Security concerns:** API keys must stay out of version control; the repository ignores .env and the report endpoints reject path-like names.

CHAPTER 11 – FUTURE SCOPE

- Integration of additional tools such as academic search (arXiv, Semantic Scholar) and PDF reading.
- Improved autonomous planning with per-sub-query coverage tracking.
- Multi-agent collaboration, for example parallel researcher agents per sub-query and an independent fact-checking agent using a different model.
- Long-term memory that reuses sources and findings from earlier reports.
- Improved RAG with chunking, embeddings and re-ranking so the most relevant passages, not only the first 6,000 characters, reach the model.
- Multimodal capabilities, such as reading charts and tables in sources.
- Voice-based question input.
- Better monitoring and observability through LangSmith tracing (already supported by the dependencies but disabled by default).
- Human-in-the-loop approval of the research plan before searching starts.
- Deployment as a production-ready multi-user web application with authentication and a job queue.

CHAPTER 12 – CONCLUSION

This project addressed the problem of turning a complex research question into a complete, cited report without manual searching and reading. A plain LLM answer is limited to its training data, can fabricate references and never checks its own coverage, while a search engine leaves all synthesis to the user.

We built the Autonomous Deep Research AI Agent, a single agent implemented as a LangGraph state machine. Its agentic approach combines planning by task decomposition, tool use through a web search API and a robust scraper, web retrieval-augmented generation restricted to numbered sources, and a self-reflection loop that scores each draft and turns detected gaps into follow-up searches. Control-flow decisions use validated Pydantic structured output, and hard limits on review rounds, tokens and recursion keep every run bounded. The technologies used are Python, LangGraph, Google Gemini, Tavily, httpx, trafilatura, Pydantic, Starlette and a React/TypeScript interface, with a CLI and a Streamlit app as alternatives.

The major functionality includes live progress streaming, a sources panel with scrape status, a report reader with citation previews and draft comparison, and saved reports. All 104 automated tests passed. In the live runs, 3 of 3 runs completed, with an average of 206 s and 25,394 tokens per run, and 100% of the citations in the final reports pointed to retrieved sources. In live run 1 the reviewer found that three of the five planned sub-queries were not covered by the first draft, and the follow-up round raised the self-assessed score from 6/10 to 9/10, which shows the value of the loop.

Overall, the system is effective for producing grounded first drafts that a person can verify through the cited links. Its main weaknesses are dependence on free-tier API quotas, pages that cannot be scraped and the fact that the quality score is the model's own judgement.

The key learning from the project is that reliable agents come from explicit structure: keeping the loop in a graph, validating every decision the model makes, generating references in code and testing each node with fakes made the behaviour predictable, while the LLM is used only where judgement is needed.

REFERENCES

Research Papers

[1] R. Nakano et al., "WebGPT: Browser-assisted question-answering with human feedback," arXiv preprint arXiv:2112.09332, 2021.

[2] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[3] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. International Conference on Learning Representations (ICLR), 2023.

[4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[5] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[6] Y. Shao, Y. Jiang, T. A. Kanell, P. Xu, O. Khattab and M. S. Lam, "Assisting in writing Wikipedia-like articles from scratch with large language models," in Proc. NAACL, 2024.

[7] A. Barbaresi, "Trafilatura: A web scraping library and command-line tool for text discovery and extraction," in Proc. ACL-IJCNLP: System Demonstrations, 2021.

Books

[8] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2020.

Websites, Documentation, APIs and Libraries

[9] LangChain, Inc., "LangGraph documentation," 2026. [Online]. Available: https://langchain-ai.github.io/langgraph/

[10] LangChain, Inc., "langchain-google-genai integration documentation," 2026. [Online]. Available: https://python.langchain.com/docs/integrations/chat/google_generative_ai/

[11] Google, "Gemini API documentation," 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs

[12] Tavily, "Tavily Search API documentation," 2026. [Online]. Available: https://docs.tavily.com/

[13] Pydantic, "Pydantic documentation (v2)," 2026. [Online]. Available: https://docs.pydantic.dev/

[14] Encode, "HTTPX documentation," 2026. [Online]. Available: https://www.python-httpx.org/

[15] L. Richardson, “Beautiful Soup documentation,” 2026. [Online]. Available: https://www.crummy.com/software/BeautifulSoup/bs4/doc/

[16] Encode, “Starlette documentation” and “Uvicorn documentation,” 2026. [Online]. Available: https://www.starlette.io/ and https://www.uvicorn.org/

[17] Snowflake Inc., “Streamlit documentation,” 2026. [Online]. Available: https://docs.streamlit.io/

[18] Meta Platforms, Inc., “React documentation,” 2026. [Online]. Available: https://react.dev/

[19] Vite, Tailwind CSS and shadcn/ui documentation, 2026. [Online]. Available: https://vite.dev/, https://tailwindcss.com/, https://ui.shadcn.com/

[20] pytest-dev, “pytest documentation,” and Vitest, “Vitest documentation,” 2026. [Online]. Available: https://docs.pytest.org/ and https://vitest.dev/

[21] W. McGugan, “Rich documentation,” 2026. [Online]. Available: https://rich.readthedocs.io/

[22] M. Koster, G. Illyes, H. Zeller and L. Sassman, “Robots Exclusion Protocol,” RFC 9309, IETF, 2022.

[23] Microsoft, “Playwright for Python documentation” (used only to capture the screenshots in this report), 2026. [Online]. Available: https://playwright.dev/python/

No external datasets were used; the test questions are listed in Appendix D.

APPENDIX

Appendix A – Source Code

The complete source code, the built web UI, the tests and this report are available in the project repository. Important code snippets are explained in Chapter 7.

GitHub Repository: https://github.com/thevyompatel/Autonomous-Deep-Research-AI-Agent

Deployed Link: Not deployed publicly. The application runs locally at http://127.0.0.1:8000 with `python -m app.web` (setup steps are in README.md).

Appendix B – Additional Screenshots

Figure 14 shows the library after the three live runs of Chapter 8: each saved report is listed with its age and word count, the revised report carries the "Revised" badge and its final score, and the sources panel marks scraped pages with a tick and snippet fallbacks with a warning icon.

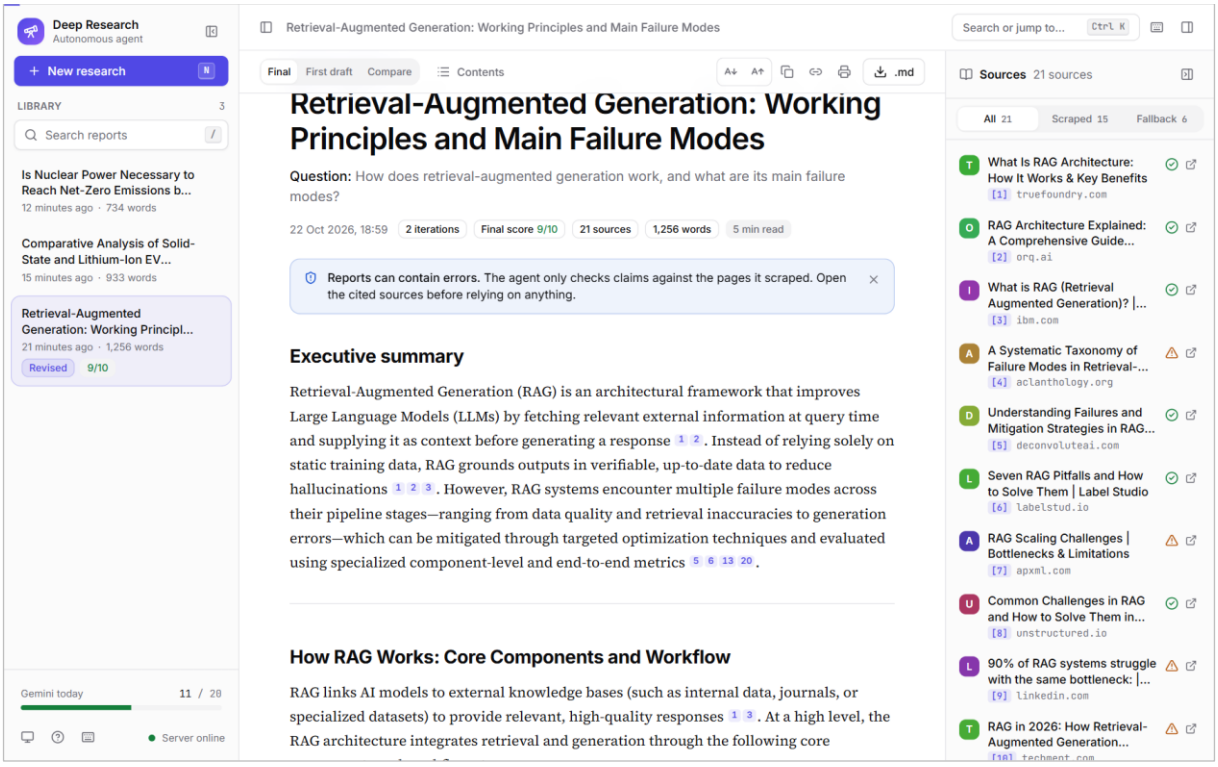


Figure 14: Report library with the three live-run reports and the sources panel

Appendix C – Sample Prompts

The three prompt templates live in src/agent/prompts.py. Table 14 gives the opening and key rules of each (abridged; placeholders in braces are filled at run time).

Table 14: Sample prompts used in the system

Sr. No.	Prompt (abridged)	Purpose
1	"You are the planning step of a web research agent. Today's date is {today}. Break the research question below into 3 to 6 web search queries ... Each query is specific ... The queries do not overlap ... When the topic is time-sensitive, include a recency term such as the year."	Planner: task decomposition into the Plan schema
2	"You are writing a research report for a reader who has not seen the sources ... Numbered sources (the only evidence you may use): {sources} ... Every factual claim ends with a citation such as [2] ... Do not add facts from memory ... write "insufficient evidence" instead of guessing."	Synthesizer: grounded, cited report writing
3	"You are a strict editor reviewing a draft research report ... check: Coverage, Support, Accuracy, Depth ... A score of 8 or more means the report is ready to publish ... In "follow_up_queries", give at most {max_follow_ups} web search queries that target only those gaps."	Reflector: critique, score and follow-up queries (Critique schema)

Appendix D – Test Data

No dataset was used. Table 15 lists the research questions. Q1-Q5 are the evaluation questions defined in app/evaluate.py, chosen to cover five question types; the questions used in the live runs of Chapter 8 are marked. The offline tests use fake search results, pages and LLM responses defined in tests/.

Table 15: Test data: research questions

ID	Type	Question	Used in live run
Q1	Factual comparison	Compare solid-state and lithium-ion EV batteries for 2026	Run 2 (CLI)
Q2	Recent news	What are the most significant AI regulation developments of 2026?	No
Q3	Technical how-it-works	How does retrieval-augmented generation work, and what are its main failure modes?	Run 1 (Web UI)
Q4	Market overview	What is the state of the global semiconductor foundry market in 2026?	No
Q5	Controversial topic	Is nuclear power necessary to reach net-zero emissions by 2050?	Run 3 (CLI)